CrewAi

1 基础介绍

Crew AI 是一个基于Python 开发的开源多智能体协作框架。该框架有João Moura于2023年创立。其核心架构包含 智能体、工具、任务和流程，支持动态任务分配与大语言模型集成，设计理念是模拟人类团队协作。主要应用于企业自动化、客服系统以及数据分析领域。

2024年，CrewAI完成1800万美元融资，投资方包括boldstart ventures、Insight Partners等，AI专家吴恩达等参与投资。

2 核心技术（四大组件）

Crew AI 框架的核心组件包括 **代理Agent、任务Task、流程Process和团队Crew，**其中Agent是执行任务、做出决策并进行通信的自治单元并可以被设计为具有特定角色、目标和工具，任务是具体的工作或者目标且可被动态分配，流程是协调Agent执行任务的方式，团队由多个Agent组成的协作团队，工具是Agent执行任务所使用的技能。

代码块

```
from crewai import Agent, Task, Crew, Process
# 更丰富的
from crewai import (
    LLM, BaseLLM,               # 模型：显式的模型对象 + 抽象基类
    Agent,                      # 智能体
    Task, TaskOutput,           # 任务 + 任务产物
    Crew, CrewOutput,           # 团队 + 团队产物
    Process,                    # 执行方式（sequential / hierarchical）
    Flow,                       # 事件驱动工作流 ← 1.x 新增
    # Flow 是在遇到"需要分支/循环/状态机"时才上场，官方推荐的架构是 Flow 做外层流程，内层嵌 Crew
    tools,
    Knowledge, Memory,          # 知识库 + 统一记忆 ← 1.x 新增
    LLMGuardrail,               # LLM 护栏（校验输出）
    PlanningConfig,             # 规划配置
    ExecutionContext,           # 执行上下文 ← 1.x 新增
    Entity, RuntimeState,       # 运行时状态 / 可序列化实体 ← 1.x 新增
)
通常来说，agent 和 task 是用yaml文件来确定的。

补充，crewai的自定义工具：
@tool 装饰器
from crewai.tools import tool

```

```
@tool("查淡旺季")
def check_season(destination: str) -> str:
    """查询某个目的地的淡旺季月份分布，用于判断经营窗口。"""
    return "7–8月、12–1月为旺季；4–6月、9–10月为平季；3月、11月为淡季"

```

（1）代理Agent

- 角色Role：智能体在团队中的身份，如“资深数据分析师”
- 目标Goal：该角色需要达成的具体目的，如”分析竞争对手的定价策略“
- 背景故事Backstory：为LLM提供上下文，使其行为更符合角色设定
- 工具Tools:智能体可以调用的能力集，如网格搜索、文件读写、代码执行等

（2）任务Task

- 描述Description：对任务的清晰说明
- 预期输出Expected Output：定义任务完成后的具体成果格式，如”一份包含***飞书文档“
- 分配对象Agent:指定由哪一个智能体负责此任务

（3）流程Process

- 顺序Sequential：**任务按列表顺序一个接一个地完成**
- 层级Hierachical：由一个”经理“智能体动态地将任务委派给其他”员工“智能体
- 自定义Custom：开发者可以实现自己的复杂协作逻辑

（4）团队Crew

- 这是将所有Agent和Task组织在一起的容器
- 它定义了团队的协作流程Process，例如是顺序执行、层级委派还是并行处理?

框架的关键特性包括 **基于角色的Agent设计、自主的Agent间的任务委托、灵活的任务管理、流程驱动（支持顺序和层级流程）**、可将输出保存为文件、可解析输出为Pydantic或者JSON格式以及与开源模型兼容并支持通过工具集成本地模型。

Crew AI 的设计理念是模拟人类团队协作，通过角色分工与信息共享共同解决复杂问题，相较于其他多智能体框架(如AutoGen、ChatDev)，其旨在结合Autongen的对话灵活性与ChatDev的结构化流程，同时避免流程的僵化，流程设计更具动态性和适应性以支持生产和开发工作流

2.1 Agent

2.2 Process

```
代码块 process = sequential | hierarchical
```

（1）process = sequential 顺序执行

（2）

代码块

```
def _run_hierarchical_process(self) -> CrewOutput:
    """Creates and assigns a manager agent to complete the tasks."""
    self._create_manager_agent()      # ← 唯一的差别
    return self._execute_tasks(self.tasks)

manager是怎么造出来的?
manager = Agent(
    role=i18n.retrieve("hierarchical_manager_agent", "role"),      # ← 内置模板
    goal=i18n.retrieve("hierarchical_manager_agent", "goal"),
    backstory=i18n.retrieve("hierarchical_manager_agent", "backstory"),
    tools=AgentTools(agents=self.agents).tools(),   # ★ 关键在这行
    allow_delegation=True,
    llm=self.manager_llm,
    verbose=self.verbose,
)
是怎么干活的?     靠 function calling
manager 思考 → 决定"这活给竞品分析师" → 输出一个 tool_call（调"竞品分析师"这个工具）
                                        ↓
                                框架执行这个工具 = 让竞品分析师真的去干活
                                        ↓
                                干完的结果作为 function_response 回灌给 manager

```

	工具装给谁	谁决定派活
sequential	每个执行任务的 agent	agent 自己（干到一半发现该别人干，自己转交）
hierarchical	manager	manager（统一调度）

2.3 Task （_execute_tasks）

2.4 Flow

Crew 是"一条流水线"，Flow 是"一张流程图"。

官方给的四条价值：

1. 简化工作流创建——把多个 Crew 和 task 串成复杂工作流

2. 状态管理——在流程内共享 state
3. 事件驱动架构
4. 灵活控制流——条件、循环、分支

关键定位：Flow 编排 Crew，而**不是取代它**。

代码块

```
from crewai.flow.flow import Flow, start, listen, router, or_, and_
```

一个flow 的demo

代码块

```
"""CrewAI Flow 版 —— 同一件分析任务，改用「并行 + 汇合」的编排。

三处关键差异:
    ① 两个 @start 方法 → 流程一开始就同时跑（Crew 的 sequential 做不到）
    ② @listen(and_(a, b)) → 两个都完成了才触发写报告（汇合）
    ③ self.state → 三个步骤共享一块状态，不用手动拼 context

跑法: python demp_flow.py
"""
register_after_llm_call_hook(log_after)

# ============================================================
# 三、三个 Agent（分工）
# ============================================================
from crewai import Agent

researcher = Agent(
    role="出境游市场研究员",
    goal="摸清欧洲跟团游市场的规模、趋势与淡旺季规律",
    backstory="你在大型出境游批发商做了八年市场研究，讲证据不堆形容词。",
    llm=llm,
    verbose=True,
)

analyst = Agent(
    role="竞品分析师",
    goal="拆解欧洲跟团游主要玩家的产品结构、价格带与差异化打法",
    backstory="你是竞品分析出身，总能指出各家真正的差异点在哪里。",
    llm=llm,
    verbose=True,
)

```

```
writer = Agent(
    role="分析报告撰写人",
    goal="把市场与竞品的研究结果收敛成给业务负责人看的结论",
    backstory="你写报告结论先行、证据随后，最讨厌正确的废话。",
    llm=llm,
    verbose=True,
)
```

代码块

```
# ============================================================
# 四、Flow：共享状态 + 三个步骤
# ============================================================
from pydantic import BaseModel

from crewai import Crew, Task
from crewai.flow.flow import Flow, and_, listen, start

class TourState(BaseModel):
    """流程的共享状态 —— 三个步骤都读写它，不用手动拼 context。"""

    destination: str = "欧洲"
    market_notes: str = ""
    competitor_notes: str = ""
    report: str = ""

class TourFlow(Flow[TourState]):

    @start()
    def market_research(self):
        """第一站：市场研究。和下面的竞品研究同时开跑。"""
        result = Crew(
            agents=[researcher],
            tasks=[
                Task(
                    description=(
                        f"研究{self.state.destination}跟团游市场的淡旺季规律，"
                        "以及它对经营策略的启示。"
                    ),
                    expected_output="一段 200 字以内的简短总结，直接给结论。",
                    agent=researcher,
                )
            ],
```

```
        ).kickoff()
        self.state.market_notes = result.raw

    @start()
    def competitor_research(self):
        """第二站：竞品分析。和上面的市场研究同时开跑。"""
        result = Crew(
            agents=[analyst],
            tasks=[
                Task(
                    description=(
                        f"分析{self.state.destination}跟团游市场的主要玩家类型与真正的差异点。"
                    ),
                    expected_output="一段 200 字以内的简短总结，直接给结论。",
                    agent=analyst,
                )
            ],
        ).kickoff()
        self.state.competitor_notes = result.raw

    @listen(and_(market_research, competitor_research))
    def write_report(self):
        """等上面两个都完成，才轮到写报告（汇合）。"""
        result = Crew(
            agents=[writer],
            tasks=[
                Task(
                    description=(
                        f"综合下面两份材料，写出{self.state.destination}跟团游的"
                        "市场与竞品分析结论。\n\n"
                        f"【市场研究】{self.state.market_notes}\n\n"
                        f"【竞品分析】{self.state.competitor_notes}"
                    ),
                    expected_output="一段 300 字以内的总结，结论先行。",
                    agent=writer,
                    output_file="output/flow_report.md",
                )
            ],
        ).kickoff()
        self.state.report = result.raw
        return result.raw  # 最后一个方法返回的，就是 flow.kickoff() 的结果


```

代码块

```python
# ============================================================
# 五、开跑
# ============================================================
if __name__ == "__main__":
    flow = TourFlow()

    # 生成流程拓扑图（HTML，浏览器打开能看到两个 @start 并排汇合到 write_report）
    try:
        flow.plot("flow_plot")
        print("[已生成流程拓扑图 flow_plot.html]\n")
    except Exception as exc:  # 画图失败不影响主流程
        print(f"[plot 跳过: {exc}]\n")

    final = flow.kickoff(inputs={"destination": "欧洲"})

    print("\n" + "=" * 72)
    print("最终报告")
    print("=" * 72)
    print(final)
    print("=" * 72)
    print(f"共享状态里的市场研究（前 80 字）: {flow.state.market_notes[:80]}")
    print(f"共享状态里的竞品分析（前 80 字）: {flow.state.competitor_notes[:80]}")
```

2.5 tools

2.6 get_context()

每一个任务执行前，拿到的context是什么？是前序任务的最终输出，只有结论，没有过程。并且只在这一次 `kickoff` 内有效。

2.7 防爆token机制 & hook

```
Tracing Status
Info: Tracing is disabled.

To enable tracing, do any one of these:
• Set tracing=True in your Crew/Flow code
• Set CREWAI_TRACING_ENABLED=true in your project's .env file
• Run: crewai traces enable
```

开 `tracing=True`，CrewAI 会把每次执行记录下来

3 快速入门

https://docs.crewai.com.cn/en/quickstart

场景一：自动化内容创作

场景二：市场与竞品分析

> 一个 Crew、三个 Agent、三个 Task、Sequential 流程——把 CrewAI的四件套（Agent / Task / Crew / Process）用最短的路径跑通。

目录结构

```
CrewAI_learning/
├── main.py                    # 入口：造 crew、注入参数、kickoff
├── requirements.txt
├── .env                       # 环境配置
├── output/                    # 三个任务的产出落在这里（自动创建）
└── tour_analysis/             # crew 包
    ├── __init__.py
    ├── crew.py                # @CrewBase 类：把 Agent 和 Task 组装成 Crew
    └── config/
        ├── agents.yaml        # 三个角色：role / goal / backstory
        └── tasks.yaml         # 三个任务：description / expected_output / agent / context
```

执行顺序

```
crew.kickoff(inputs={...})
  │
  ├─ ① 检查点：有 from_checkpoint 就直接恢复续跑（短路返回）
  │ #解释 checkpoint 就是存档点，一条流水线有 10 个任务，跑到第 8 个时网络断了。
  │  #重跑要从第 1 个开始，前面 2 小时的活全白干。有了这个checkpoint ，就能从第8个接着干。
  ├─ ② 准备输入：prepare_kickoff(self, inputs, ...)   # 把 {destination} 等占位符插值进去
  │
  ├─ ③ 按 process 分发：                                  # ← Process 在这里起作用
  │     (1) Process.sequential   → _run_sequential_process() --> # 直接开跑
                  def _run_sequential_process(self) -> CrewOutput:
                      return self._execute_tasks(self.tasks)          # 直接开跑
  │     (2) Process.hierarchical → _run_hierarchical_process()  --> # 先造manager agent,然后再跑
                def _run_hierarchical_process(self) -> CrewOutput:
                      self._create_manager_agent()                    # 唯一的区别：先造一个"经理"
                      #层级流程，其他所有的agent都是这个经理的工具，经理负责派活
                      return self._execute_tasks(self.tasks)
  │
```

```
├─ ④ 任务循环：_execute_tasks(self.tasks)                    # ★ 引擎①：任务编排
│       for task_index, task in enumerate(tasks):
│           context = self._get_context(task:当前要执行的任务，task_outputs：到目前为止已经跑完的任务的输出列表)    # 前序任务的输出 → 上下文
            # 红色这一行是任务之间传数据的唯一通道
│           if task.async_execution: 推入 futures 并行跑
│           else:
│               task.execute_sync(agent=..., context=..., tools=...)
│                   └─ ⑤ agent.execute_task(task, context, tools)
│                           ├─ _prepare_task_execution()    # role/goal/backstory + 任务描述 + 记忆 → prompt
│                           └─ agent_executor.invoke({...}) # ★ 引擎②：ReAct / function calling 循环
│                                   ├─ 调 LLM
│                                   ├─ 有工具调用 → 执行 → 结果回填 → 再调 LLM
│                                   └─ 无工具调用 → 得到最终答案
│                   └─ 组装 TaskOutput → 跑 guardrail → 写 output_file → 发 TaskCompletedEvent
│
├─ ⑥ 汇总：_create_crew_output(task_outputs) → CrewOutput
│
└─ ⑦ finally: _drain_memory_writes() / clear_files()    # 隐式收尾，不用你管


补充 agent_executor ：
class AgentExecutor(Flow[AgentExecutorState], BaseAgentExecutor):
    """Agent Executor for both standalone agents and crew-bound agents.
    Inherits from:
    - Flow[AgentExecutorState]: Provides flow orchestration capabilities
    - BaseAgentExecutor: Provides memory methods (short/long/external term)
    ...
    """
agent_executor 的默认实现类 AgentExecutor 继承自 Flow
```

代码块

```python
# ① 定义 Agent（纯声明，此时不跑任何东西）
from crewai import Agent
researcher = Agent(
    role="出境游市场研究员",
    goal="摸清欧洲跟团游市场的规模、趋势与客群结构",
    backstory="你在大型出境游批发商做了八年市场研究，讲证据不堆形容词。",
    llm="gpt-4o-mini",
    verbose=True,
)

```

```python
# ② 定义 Task（也是一份声明，此时不跑）
from crewai import Task
research_task = Task(
    description="研究欧洲跟团游市场：规模、客群、外部因素、淡旺季，给出判断依据。",
    expected_output="一份 Markdown 研究笔记，四个小节，每节有明确结论。",
    agent=researcher,                          # ← 谁来做
    output_file="output/01_research.md", # ← 做完自动落盘（Task 自己写，不是 Crew）
)

# ③ 定义 Crew —— 把 agents / tasks / process 绑在一起（这就是"执行引擎的载体"）
from crewai import Crew, Process
crew = Crew(
    agents=[researcher],
    tasks=[research_task],
    process=Process.sequential,           # 调度模式（默认值，可省）
    verbose=True,
)

# ④ 开跑 —— 入口唯一，一句话把整条流水线跑完
result = crew.kickoff(inputs={"destination": "欧洲"})

# ⑤ 拿结果（ADK 拿到的是一条事件流，CrewAI 拿到的是一个结果对象）
print(result.raw)                         # 最终文本
print(result.tasks_output)            # 每个任务的产出，按顺序排列
# 没有第 ⑥ 步收尾 —— kickoff 内部 finally 会自己清理（memory 落盘、临时文件等）
```

crewAI具体是怎么执行的呢?

```
crew.kickoff(inputs={"destination": "欧洲"})                    # demp.py:73
│
├─ ① 检查点：crew 没有 from_checkpoint → 直接跳过
│
├─ ② 准备输入：prepare_kickoff(crew, {"destination": "欧洲"}, None)
│     └─ 作用是把 {xxx} 占位符插值进 agent / task 的文本里
│        ⚠️ 你的 goal / description 都是写死的"欧洲"，压根没有 {destination}
│           → 这一步什么都没替换，inputs 白传了（不报错，但也用不上）
│
├─ ③ 按 process 分发：crew.process == Process.sequential
│     └─ _run_sequential_process()
│           └─ return self._execute_tasks(self.tasks)          # self.tasks 长度 = 1
│
├─ ④ 任务循环：_execute_tasks([research_task])
│     ★ 只有 1 个任务，所以这个 for 只转一圈
│
```

```
|    for task_index=0, task=research_task in enumerate(tasks):
|
|        context = self._get_context(research_task, [])
|          └─ task_outputs 此刻是空的 → context = ""（它是第一个任务，没有前置输出）
|
|        research_task.async_execution = False → 走 else 分支
|
|        research_task.execute_sync(agent=researcher, context="", tools=[])
|        |
|        ├─ self.start_time = now()
|        └─ _execute_core(...)
|             ├─ agent = researcher（就是它自己）
|             ├─ 发 TaskStartedEvent
|             └─ ★ researcher.execute_task(task=research_task, context="", tools=[])
|                      |
|                      ├─ _prepare_task_execution()
|                      |    └─ 拼 prompt，大致是：
|                      |         「出境游市场研究员」(role)
|                      |       +「摸清欧洲跟团游市场的规模、趋势与客群结构」(goal)
|                      |       +「你在大型出境游批发商做了八年…」(backstory)
|                      |       + research_task.description
|                      |       +「期望输出：一份 Markdown 研究笔记，四个小节…」(expected_output)
|                      |         （具体模板措辞由 CrewAI 决定）
|                      |
|                      └─ agent_executor.invoke({...})                ★ 引擎②
|                           ├─ 第 1 轮：调 LLM → 发到 ada 网关，模型 qwen3.8-flash
|                           |    research_task.tools = [] → 没有工具可用
|                           ├─ 没有工具调用 → 这轮回答就是最终答案 → 退出循环
|                           └─ 返回文本
|                      |
|                      └─ _finalize_task_execution：收尾、发 AgentExecutionCompletedEvent
|
|        回到 Task 侧：
|        ├─ 组装 TaskOutput(raw=那段文本, agent="出境游市场研究员", ...)
|        ├─ 跑 guardrail：没配 → 跳过
|        ├─ 写 output_file → output/01_research.md          ★ 文件在这里落地
|        |    (create_directory 默认 True，目录会自动建)
|        └─ 发 TaskCompletedEvent，return TaskOutput
|
|        task_outputs.append(task_output)        → 现在有 1 个元素
|
|    （循环结束，futures 为空）
```

```
59  │
60  ├─ ⑥ _create_crew_output(task_outputs) → CrewOutput
61  │      ├─ calculate_usage_metrics()（统计 token 用量）
62  │      └─ 发 CrewKickoffCompletedEvent
63  │
64  └─ ⑦ finally: _drain_memory_writes() / clear_files(crew.id)    ← 隐式收尾
65
66  回到 demp.py:73: result = 那个 CrewOutput
67    ├─ result.raw            → 第 76 行打印的最终文本
68    └─ result.tasks_output   → 第 77 行打印的 [TaskOutput]，长度是 1
```

标准写法

```
1   # ① YAML 里声明"内容"（不跑任何东西）
2   #    config/agents.yaml   → role / goal / backstory    ... 对应上面 ①
3   #    config/tasks.yaml    → description / expected_output / agent / context
    ... 对应上面 ②
4
5   # ② crew.py 里写"装配"（也不跑）
6   @CrewBase
7   class TourAnalysisCrew:
8       @agent
9       def market_researcher(self): return
    Agent(config=self.agents_config["market_researcher"])
10      @task
11      def market_research_task(self): return
    Task(config=self.tasks_config["market_research_task"])
12
13      @crew
14      def crew(self) -> Crew:                    # ← 这一步产出上面 ③ 那个 Crew 对象
15          return Crew(agents=self.agents, tasks=self.tasks,
    process=Process.sequential)
16
17  # ③ main.py 里开跑
18  crew = TourAnalysisCrew().crew()                                  # 车间装配 → 拿到 Crew
19  result = crew.kickoff(inputs={"destination": "欧洲"})    # 对应上面 ④
20
21  # ④ 拿结果
22  result.raw
```

一个关键的补充：Flow

场景三：软件开发辅助

4 CrewAI总结

框架的关键特性包括

- **基于角色的Agent设计：**role / goal / backstory Agent三件套
- **自主的Agent间的任务委托：**委托指的是 一个 agent 干着干着，自己判断"这活该别人干"，于是转给同事。如果是层级，就是manager 判断交给哪一个agent。
- **灵活的任务管理**：Task设计：description 、expected_output、agent(谁做)、tools(限用哪些工具)
- **流程驱动（支持顺序和层级流程）**： process = sequential | hierarchical
  - "层级流程" = 造一个 agent，把其他 agent 变成它的工具。
- 可将输出保存为文件、可解析输出为Pydantic或者JSON格式
- 与开源模型兼容：可以用LietLLM 用其他模型

与Google ADK 的不同：

- ADK 的隐喻是运行时：Agent 是进程，Session 是内存，Event 是系统调用，编排单位是一个LLM往返，在LLM之间传递的是 **事件 + 会话（Event / Session）**
- CrewAI 的隐喻是项目组：Agent 是员工，Task 是工单，Crew 是项目，编排单位是一个Task，在LLM之间传递的是 **任务 + 协作（Task / context / delegation）**

CrewAI 让你用"派活"的方式组织 agent，ADK 让你用"事件"的方式观察 agent。前者像项目经理，后者像系统工程师。

三个场景，最能看出差别

① 想看它在干什么

- ADK：async for event in runner.run_async(...) —— 每一步都在你手上
- CrewAI：result = crew.kickoff() —— 一个结果对象；要看过程得挂 LLM hook 或开 tracing

② 想接着上一轮聊

- ADK：session_id 一传，历史自动都在
- CrewAI：kickoff 是新项目；要连续必须上 Flow 的 ChatState

③ 想让人去干活

- ADK：sub_agents 树，父子之间 transfer_to_agent（控制权转移，可多层）
- CrewAI：Task 流水线 + 委托（一层经理制，或 agent 之间互相转交）

但本质底层机制都是一样的(所有的Agent框架都要有的)

一样的地方	ADK	CrewAI
Agent 定义	都得写 你是谁，instruction、背景等等	
tools	自定义工具的写法都一样，def 工具名():	
干活的循环	调模型 → 要工具就执行 → 再调模型 → 直到最终答案	完全一样
都能组队	sub_agents（上下级，可以一层层转交）	Crew(agents=[...])（团队，可以互相委托
都能把流程	ADK：Workflow（节点 + 连线）	CrewAI：Flow（@start / @listen）
事后能查的口子	ADK：Session 里一条条 Event	CrewAI：LLM hook、tracing、@persist